BreakfastWise — Product Requirements Document (PRD)

JJCET AI-PDLC Hackathon · 24–25 October 2026 · Owner: BA · Metrics: QA · Source: MRD

Product in one line: BreakfastWise tells a hotel's head chef, at 9 PM, how many guests will eat breakfast tomorrow and how much idli, dosa, pongal, poori and sambar to prepare, and shows how every number was calculated.

Stack: Streamlit (Python), loading the CSVs in `data/`. No database, no login, no external APIs.

Global rule: every screen shows a visible banner **"SYNTHETIC DEMO DATA — NOT REAL HOTEL DATA"**. All evaluation results are labelled as a synthetic simulation.

1. Features (MoSCoW)

Must-Have (3 features, the maximum)

F1 — Tomorrow's covers forecast, with the calculation shown (needs N1, N3)

Inputs come from `tomorrow_inputs.csv`: the forecast date, its weekday, `guests_included`, `guests_other`, `group_event` and `event_expected_covers`.

1. Take the **last four matching weekdays** in `daily_history.csv` before the forecast date.
2. Compute **pooled** capture rates: - `r_i = Σ covers_included ÷ Σ guests_included` - `r_o = Σ covers_other ÷ Σ guests_other`
3. Compute **resident covers** = `guests_included × r_i + guests_other × r_o`.
4. Compute **event uplift** = `event_expected_covers` (a fixed number the chef can edit; 0 if there is no event).
5. **Covers forecast** = round(resident covers + event uplift) to a whole number.
6. **No double-counting.** Historical `event_covers` are recorded separately and are **never** included in `r_i` or `r_o`. Event attendees enter the forecast only through the uplift.
7. **If fewer than 4 matching weekdays exist:** use those available and show a warning. If none exist, or a guest group has zero guests in the window, show an error instead of a number. If tomorrow has zero guests in one group, that group simply adds 0 covers; this is not an error.
8. **If the guest split is missing**, meaning any of `guests_included`, `guests_other`, `covers_included` or `covers_other` is absent from the data: show an error naming the missing columns, and show no forecast. There is **no** single-rate fallback in this version (see Won't-Have).
9. **Calculation panel:** shows the four dates used, each Σ, `r_i`, `r_o`, resident covers, uplift and the final forecast.

F2 — Prep quantities for the five dishes, with a buffer slider (needs N2, N3)

1. **Per-cover rate** for each dish = `Σ consumed ÷ Σ covers_total` over the **last 28 days** of history, **leaving out that dish's stock-out days** (on those days consumption understates demand; MRD A7).

2. **Base quantity** = covers forecast × per-cover rate.
3. **Recommended quantity** = base × (1 + buffer). The buffer slider defaults to **10%**, ranges **0–25%** and moves in 1% steps.
4. Round **up** to the unit step:

Dish	Unit	Rounding step
Idli	pieces	1
Dosa	pieces	1
Poori	pieces	1
Pongal	kg	0.1
Sambar	litres	0.5

5. Show a table: dish | unit | per-cover rate | days used | base | buffer | recommended.

F3 — 90-day history chart with an accuracy and waste/shortage panel (needs N4)

1. **Chart:** for a selected dish, show daily prepared vs. consumed vs. discarded over the 90 days, with stock-out days marked.
2. **Accuracy (MAPE):** - Use the last 14 history days. - For each day D, forecast covers using **only information available by 9 PM on D−1** (no leakage). - **Event uplift in the evaluation:**
    - The dataset has no recorded pre-breakfast event estimate.
    - If D is flagged `group_event = 1`, the uplift is the **mean** `event_covers` **of event days before D** (0 if there were none). This assumes the event booking itself is known the night before (MRD A3).
    - D's own `event_covers` is used **only as part of the actual value being scored**, never as a forecast input.
    - MAPE = mean(|forecast − actual| ÷ actual) × 100.
    - Days where actual = 0 are skipped, and the panel reports how many were skipped.
    - Show the per-day table as well as the MAPE.
3. **Waste vs. shortage simulation:** - Covers every history day that has at least 28 days of history before it. - For each dish, rebuild that day's recommendation using only data from earlier days. Use the F1 and F2 logic, the current buffer, and the same leak-free event uplift as F3.2. - **Simulated waste** = `max(0, recommended − consumed)`. - **Simulated shortage** = `max(0, consumed − recommended)`. - **Stock-out days are left out**, because true demand is unknown. - **Show one row per dish, never a total across dishes.** Pieces, kg and litres can't be added together, and there is no reliable cost data for a money-based total (see Won't-Have). Each row shows, in that dish's own unit and over the same days:

Column	Definition
Days used / excluded	Count of days in the simulation / count left out
Recorded discarded	Σ discarded
Recorded waste %	Σ discarded ÷ Σ prepared
Simulated waste	Σ simulated waste
Simulated waste %	Σ simulated waste ÷ Σ recommended
Simulated shortage	Σ simulated shortage
Simulated shortage %	Σ simulated shortage ÷ Σ consumed

- Report waste **and** shortage together for each dish. Claim a reduction for a dish only if its numbers show one.
- **Caption under the table:** "Stock-out days are excluded because true demand is unknown on those days. These are the days the chef under-prepared. Excluding them can bias the recorded waste % and limits how representative it is of all days."

Should-Have

- **F4 — Chef override (need N5).** The chef can type a covers number that replaces the forecast. F2 quantities recalculate from it. The screen shows "Overridden: forecast was X". Build this only if the Must-Haves pass first.

Could-Have (only if time is left after F4)

- CSV upload of real history in the same columns.
- A holiday flag that works like the event uplift.

Won't-Have (this hackathon)

- Machine learning models.
- Weather data or any external API.
- Rotating menus (all 5 dishes are served daily).
- Forecasts more than one day ahead (7-day view).
- Login and user accounts.
- PMS or POS integration.
- A per-weekday dish mix.
- A single-rate forecasting fallback for hotels without the included / non-included split. The single-rate figure is used only as the **evaluation baseline** in the MRD, not as a feature.
- A combined waste figure across dishes, including any money-based waste measure. The dataset has no reliable cost-per-unit data, and we will not invent prices.

2. Proposed screen layout (for UI/UX to confirm)

1. **Banner:** synthetic-data label, forecast date and weekday.
2. **Tomorrow card (F1):** a big covers number, then "Show calculation" (expander).

3. **Prep table (F2):** the buffer slider above it, then the five dishes.
4. **History tab (F3):** dish selector, chart, accuracy panel, and a waste vs. shortage table with one row per dish. Each row is in its own unit and has its own percentages; the table has no totals row.

3. Success metrics (all measured on synthetic data)

ID	Metric	Target	How QA measures it
M1 (MRD T1)	Forecast accuracy	MAPE ≤ 10% on the last 14 days, with no leakage. This is a project target, not a guarantee	Read the F3 accuracy panel; recompute 2 days by hand
M2 (MRD T4)	Waste vs. shortage	Simulated waste % and shortage % shown beside recorded waste %, separately for each of the 5 dishes, with no cross-dish total	Read the F3 panel; check 1 day by hand. Report the result honestly, even if it shows no reduction
M3 (MRD T3)	Planning speed	Chef gets the covers forecast and all 5 quantities in < 60 s from app open	Stopwatch, 3 runs, by someone who hasn't used the app
M4 (MRD T2)	Improvement over baseline	Two-rate MAPE lower than a single blended capture rate over the same days and the same leak-free rules. If it isn't, report that	Run `docs/evidence/verify_mrd_metrics.py`. This is an evaluation check only; no UI feature is added for it

> **Disclaimer.** The synthetic data was generated with the same structure the model assumes: two capture rates and stable weekday patterns. A good M1 score therefore shows the logic is implemented correctly. It does **not** show that the method works in a real hotel.

4. Out of scope and risks

- **Risk:** the two-rate method depends on hotels recording covers split by included and non-included guests (MRD A1, A3). This is unverified.
- **Risk:** there are only 6 event days in 90, so the uplift is barely exercised by the history. Only 3 of them fall in the period that can be evaluated. The demo should show the uplift by editing tomorrow's event covers.
- **Event-evaluation limitation:**
- The dataset has no recorded pre-breakfast event estimates.
- For past event days, the evaluation therefore uses the mean `event_covers` of earlier event days (0 if none) as a stand-in (F3.2). This assumes the booking itself was known the night before (MRD A3).
- Only 3 event days fall in the period that can be evaluated, so this does **not** validate the event uplift.
- Once in use, the app should save the chef's nightly `event_expected_covers`, so future evaluations can use the real estimate. That save is not a Must-Have in this hackathon.